

dataloader/kuaiREC.py (Q&A Documentation)

Q: What does this file implement at a high level? **A:** This file defines the PyTorch Dataset/-DataLoader wrapper for the preprocessed KuaiRec pickle. It converts pandas columns into tensors, casts dtypes based on the schema, and yields (features\{\}_dict, label) batches.

Q: Why is dtype handling critical here? **A:** Embedding layers require integer indices, while continuous features and masks require float tensors. Wrong dtypes can crash forward passes or silently break training.

Q: What does this file export (public API)? **A:** The public API is the set of classes/functions other modules import (sometimes re-exported by package initializers). The key top-level definitions detected here are:

```
class KUAIRECDataset # methods: __init__, format, __getitem__, __len__
class KUAIRECDataloader # methods: __init__, __getitem__
```

Q: What are the main dependencies (imports) and why do they matter? **A:** Imports show whether this file is model code (torch), data code (pandas), or evaluation/analysis (sklearn). They also determine runtime requirements.

```
import numpy as np
import pandas as pd
import pickle
import os
import torch
from torch.utils.data import Dataset, DataLoader
```

```
class KUAIRECDataset(Dataset):
    """
    Load a KuaiRec Dataset
    """
    def __init__(self, dataset_name, df, description, device):
        super(KUAIRECDataset, self).__init__()
        self.dataset_name = dataset_name
        self.df = df
        self.length = len(df)
        self.name2array = {name: torch.from_numpy(np.array(list(df[name])).reshape([self.length, -1]))).
                           for name in df.columns}
        self.format(description, device)
        self.features = [name for name, size, type in description if type != 'label']
        self.label = 'play_time'
```

```

def format(self, description, device):
    for name, size, type in description:
        if type == 'spr' or type == 'seq':
            self.name2array[name] = self.name2array[name].to(torch.long)
        elif type == 'ctn' or type == 'seqm' or type == 'other':
            self.name2array[name] = self.name2array[name].to(torch.float32)
        elif type == 'label':
            pass
        else:
            raise ValueError('unkwon type {}'.format(type))

def __getitem__(self, index):
    return {name: self.name2array[name][index] for name in self.features}, \
        self.name2array[self.label][index].squeeze()

def __len__(self):
    return self.length

class KUAIRECDatasetLoader(object):
    """
    Load KUAIRANK16DatasetLoader for torch train/eval

    :param dataset_path: dataset path
    """

    def __init__(self, dataset_name, dataset_path, device, bsz=32):

```

Q: What is KUAIRECDataset and what responsibility does it have? **A:** Load a KuaiRec Dataset

Q: What are the important methods on KUAIRECDataset? **A:** In this repo, methods like forward/loss/predict define computation and training targets. The methods found are:

```

__init__
format
__getitem__
__len__

```

Q: What does the implementation look like for KUAIRECDataset? **A:** Excerpt from the file:

```

class KUAIRECDataset(Dataset):
    """
    Load a KuaiRec Dataset
    """
    def __init__(self, dataset_name, df, description, device):
        super(KUAIRECDataset, self).__init__()
        self.dataset_name = dataset_name
        self.df = df
        self.length = len(df)
        self.name2array = {name: torch.from_numpy(np.array(list(df[name])).reshape([self.length, -1])).to(device) \
                            for name in df.columns}
        self.format(description, device)
        self.features = [name for name, size, type in description if type != 'label']

```

```

        self.label = 'play_time'

    def format(self, description, device):
        for name, size, type in description:
            if type == 'spr' or type == 'seq':
                self.name2array[name] = self.name2array[name].to(torch.long)
            elif type == 'ctn' or type == 'seqm' or type == 'other':
                self.name2array[name] = self.name2array[name].to(torch.float32)
            elif type == 'label':
                pass
            else:
                raise ValueError('unkwon type {}'.format(type))

    def __getitem__(self, index):
        return {name: self.name2array[name][index] for name in self.features}, \
            self.name2array[self.label][index].squeeze()

    def __len__(self):
        return self.length

```

Q: What is KUAIRECDataloader and what responsibility does it have? **A:** Load KUAIRANK16DataLoader for torch train/eval
:param dataset_path: dataset path

Q: What are the important methods on KUAIRECDataloader? **A:** In this repo, methods like forward/loss/predict define computation and training targets. The methods found are:

```

__init__
__getitem__

```

Q: What does the implementation look like for KUAIRECDataloader? **A:** Excerpt from the file:

```

class KUAIRECDataloader(object):
    """
    Load KUAIRANK16DataLoader for torch train/eval

    :param dataset_path: dataset path
    """

    def __init__(self, dataset_name, dataset_path, device, bsz=32):
        assert os.path.exists(dataset_path), '{} does not exist'.format(dataset_path)
        with open(dataset_path, 'rb+') as f:
            data = pickle.load(f)
        self.dataset_name = dataset_name
        self.dataloaders = {}
        self.description = data['description']
        for key, df in data.items():
            if key == 'description':
                continue
            self.dataloaders[key] = DataLoader(KUAIRECDataset(dataset_name, df, self.description, device), batch_size=bsz, shuffle=True)
        self.keys = list(self.dataloaders.keys())

    def __getitem__(self, name):
        assert name in self.keys, '{} not in keys of datasets'.format(name)
        return self.dataloaders[name]

```

Q: Example: end-to-end data flow involving this file **A:** Core pipeline shape (schematic):

```

dataloaders = KUAIRECDataloader("kuaiREC", "./dataset/kuaiREC/kuaiREC_data.pkl", device, bsz=2048)
features, label = next(iter(dataloaders["train"]))
print(features.keys(), label.shape)

```

Q: Full source code of the Python file **A:** The complete file is included verbatim for reference.

```

import numpy as np
import pandas as pd
import pickle
import os
import torch
from torch.utils.data import Dataset, DataLoader

class KUAIRECDataset(Dataset):
    """
    Load a KuaiRec Dataset
    """
    def __init__(self, dataset_name, df, description, device):
        super(KUAIRECDataset, self).__init__()
        self.dataset_name = dataset_name
        self.df = df
        self.length = len(df)
        self.name2array = {name: torch.from_numpy(np.array(list(df[name])).reshape([self.length, -1])).to(device) \
                           for name in df.columns}
        self.format(description, device)
        self.features = [name for name, size, type in description if type != 'label']
        self.label = 'play_time'

    def format(self, description, device):
        for name, size, type in description:
            if type == 'spr' or type == 'seq':
                self.name2array[name] = self.name2array[name].to(torch.long)
            elif type == 'ctn' or type == 'seqm' or type == 'other':
                self.name2array[name] = self.name2array[name].to(torch.float32)
            elif type == 'label':
                pass
            else:
                raise ValueError('unkwon type {}'.format(type))

    def __getitem__(self, index):
        return {name: self.name2array[name][index] for name in self.features}, \
               self.name2array[self.label][index].squeeze()

    def __len__(self):
        return self.length

class KUAIRECDataloader(object):
    """
    Load KUAIRANK16Dataloader for torch train/eval

    :param dataset_path: dataset path
    """
    def __init__(self, dataset_name, dataset_path, device, bsz=32):
        assert os.path.exists(dataset_path), '{} does not exist'.format(dataset_path)
        with open(dataset_path, 'rb+') as f:
            data = pickle.load(f)
        self.dataset_name = dataset_name
        self.dataloaders = {}
        self.description = data['description']
        for key, df in data.items():
            if key == 'description':
                continue
            self.dataloaders[key] = DataLoader(KUAIRECDataset(dataset_name, df, self.description, device), batch_size=bsz, shuffle=True)

```

```
self.keys = list(self.dataloaders.keys())

def __getitem__(self, name):
    assert name in self.keys, '{} not in keys of datasets'.format(name)
    return self.dataloaders[name]
```